Course No. CSSE - 601

Distributed System

Course No. CSSE - 601

Coordination Algorithm

Objective

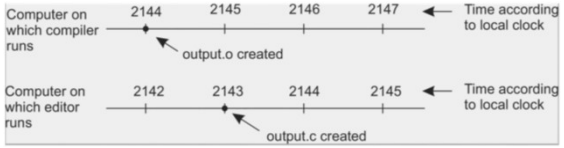
- Coordination
- Clock Synchronization
- Logical Clocks
- Lamport's Logical Clocks
- Vector Logical Clock
- Distributed Mutual Exclusion
- Single-Computer vs. Distributed System
- DME: Requirements
- DME: Performance
- DME: A Simple Solution
- DME: Lamport's Algorithm
- DME: The Ricart-Agrawala Algorithm

Coordination

- **Synchronization** and **coordination** are **two closely related phenomena**.
- In **process synchronization** we make sure that one **process waits** for another to **complete** its **operation**.

Clock Synchronization

- In a **centralized system**, time is **unambiguous**.
- A **process** want to **know the time**, it **simple makes a call** to the **operating system**.
- In a **distributed system**, **achieving agreement** on time is not so **simple**.
- If a **program** consists of 100 files, **recompile every time** because **one file** has been **changed**.



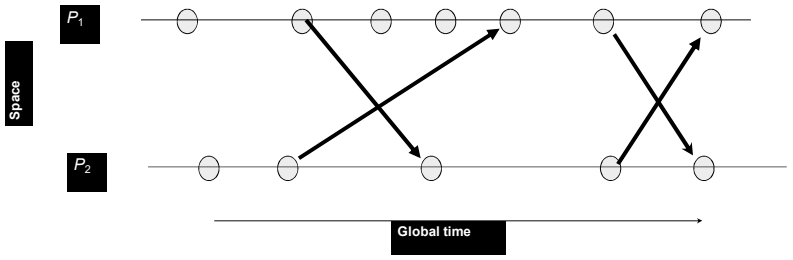
Logical Clocks

- **Lamport's Logical Clocks:** To **synchronize logical clocks**, **Lamport** define a **relation** called **happens-before.**
- The **expression** $a \rightarrow b$ is read "Event *a* **happens before event** *b*" and means that all processes **agree** that **first event** *a* **occurs**, then **afterward event b** occurs.
- The **happens-before relation** can be **observed directly** in two **situations**:
- If *a* and *b* are **events** in the **same process** and *a* **occurs before** *b*, then $a \rightarrow b$ is **true**.
- If *a* is the **event** of a **message** being **sent** by **one process**, and *b* is the **event** of the **message** being **received** by **another process**, then $a \rightarrow b$ is also **true**. A **message** cannot be **received** before it is **sent**, or even at the **same time** it is **sent** it takes a **finite**, **nonzero** amount of time to **arrive**.

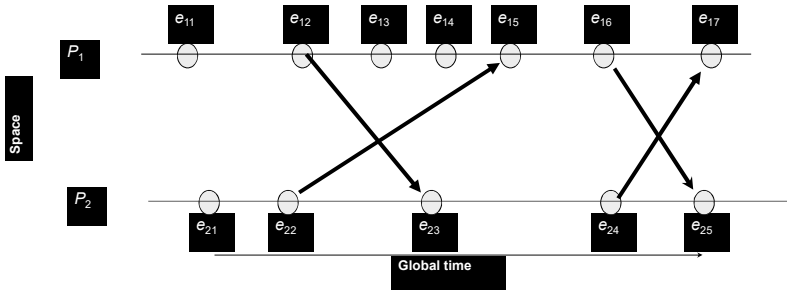
Logical Clocks

- **Lamport's Logical Clocks:**
- To **implement Lamport's logical clock** in **distributed systems** counter C_i. These counters are **updated** according to the following steps
- Before **executing** an event (i.e. **sending** a **message** over the network, **delivering** a **message** to an **application**, or some other internal event), P_i **increments** C_i: $C_i \leftarrow C_i + 1$.
- When **process** P_i send a **message** *m* to **process** P_j it sets *m's* **timestamp** *ts(m)* **equal** to C_i after having **executed** the previous step.
- Upon the **receipt** of a **message** *m*, **process** P_j adjusts its own local counter as $C_j \leftarrow \max\{C_j, ts(m)\}$ after which it then **executes** the **first step** and **delivers** the **message** to the **application**.

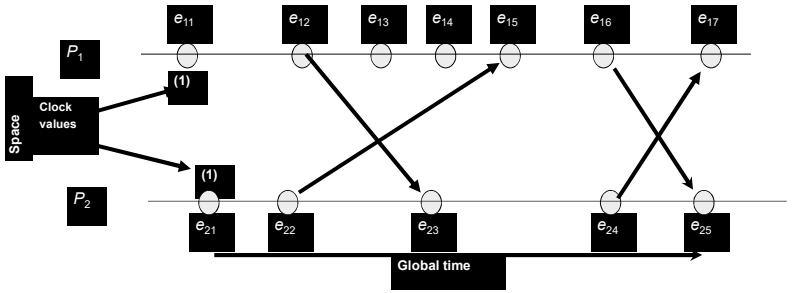
Lamport's Logical Clocks Example



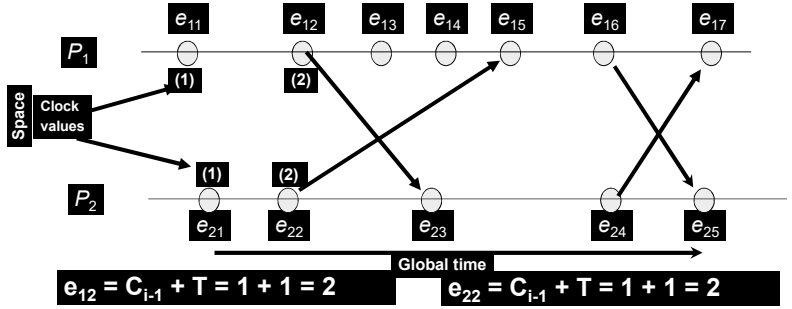
Logical Clocks Example



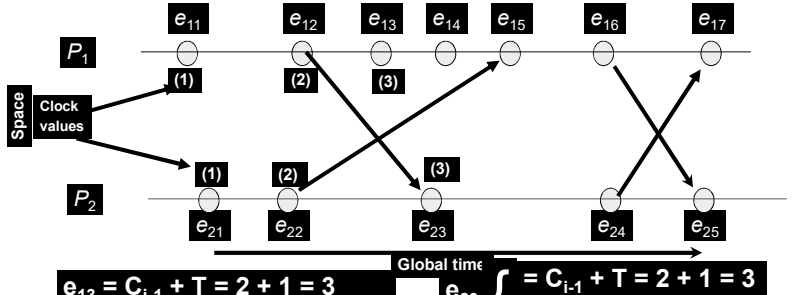
Logical Clocks Example



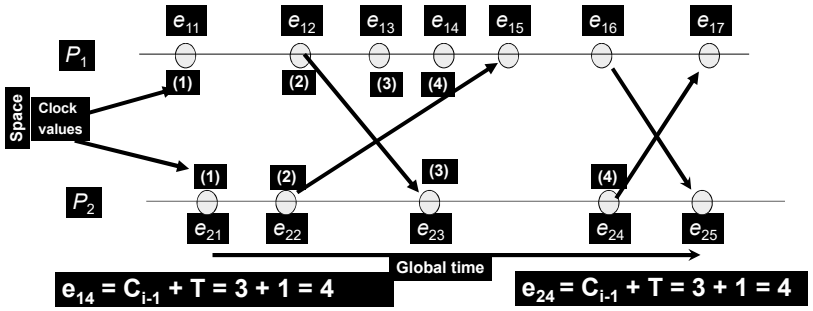
Logical Clocks Example



Logical Clocks Example



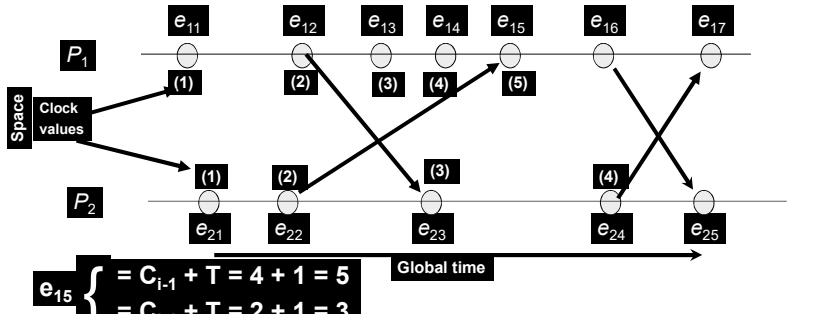
Logical Clocks Example



$e_{14} = C_{i-1} + T = 3 + 1 = 4$

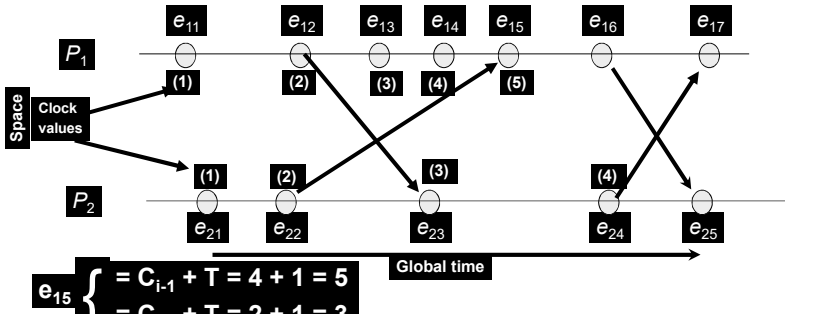
$e_{24} = C_{i-1} + T = 3 + 1 = 4$

Logical Clocks Example



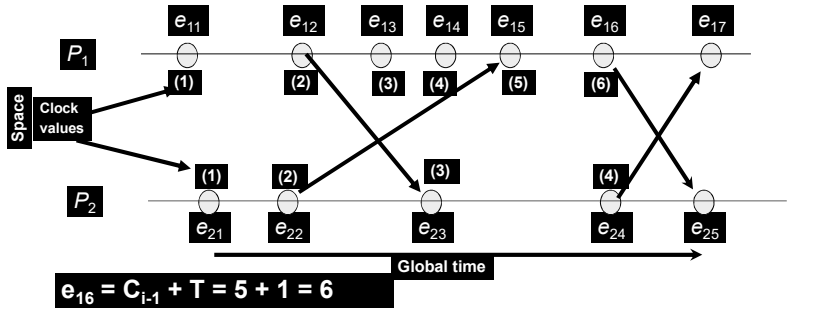
e_{15} { $= C_{i-1} + T = 4 + 1 = 5$
$= C_{j-1} + T = 2 + 1 = 3$

Logical Clocks Example



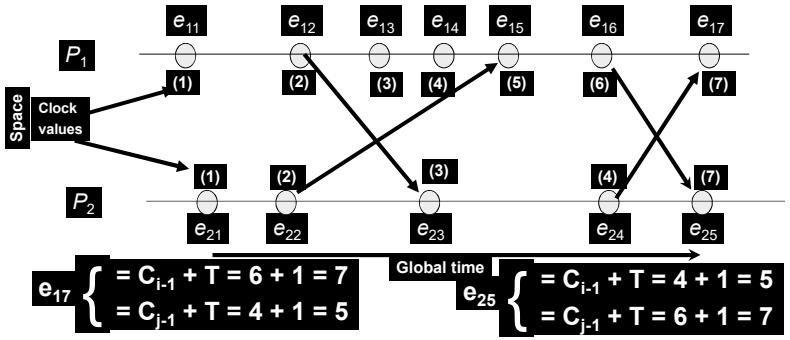
e_{15} { $= C_{i-1} + T = 4 + 1 = 5$
$= C_{j-1} + T = 2 + 1 = 3$

Logical Clocks Example



$e_{16} = C_{i-1} + T = 5 + 1 = 6$

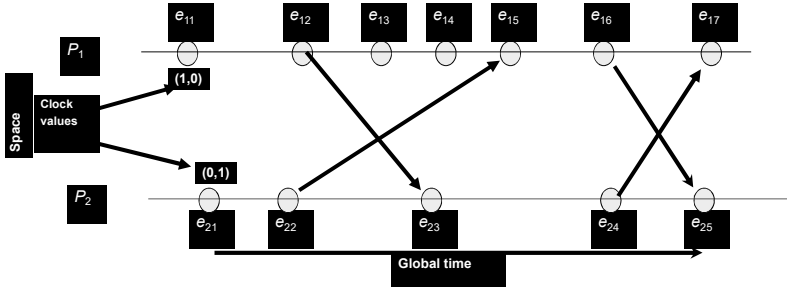
Logical Clocks Example



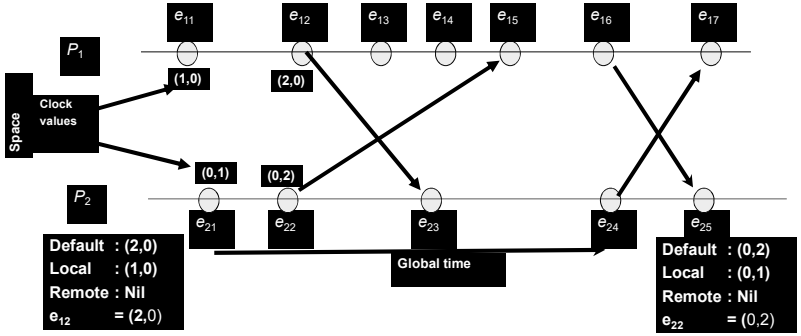
Vector Logical Clock

- The system of **vector clocks** was **independently proposed** by **Fidge** and **Mattern**:
  - Let *n* be the **number** of **processes** in a **distributed system**
  - Each **process** Pi is **equipped** with a clock *Ci*(*a*), which is an **integer vector** of **length** *n*
  - The **clock** *C*i can be **thought** of as a **function** that **assigns** a **vector** *Ci*(*a*) to any **event** *a*
  - *Ci*(*a*) is **referred** to as the **timestamp** of **event** *e*1 at *Pi. Ci*[*l*] the *i*th **entry** of Ci, **corresponds** to Pi's **own logical time**
  - *Ci*[*j*] is Pi's **best guess** of the **logical time** at Pj. More specifically, at any point in time, the *j*th entry of *Ci* **indicates** the time of **occurrence** of the last event at Pj which "**happened before**" the current point in time at Pi
  - This "**happened before**" relationship could be **established** directly by **communication** from Pj to Pi or indirectly through **communication** with other **processes**.

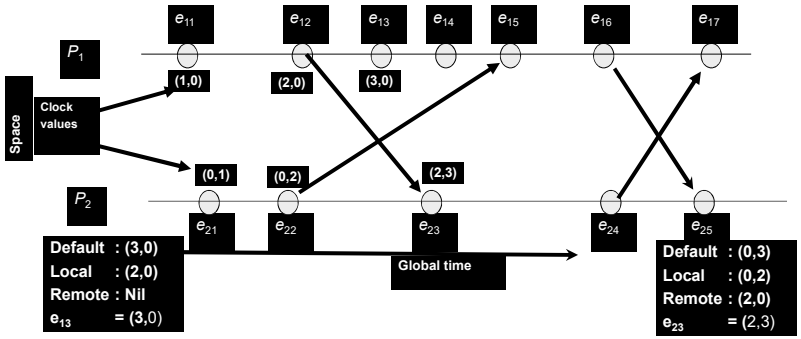
Vector Logical Clock Example



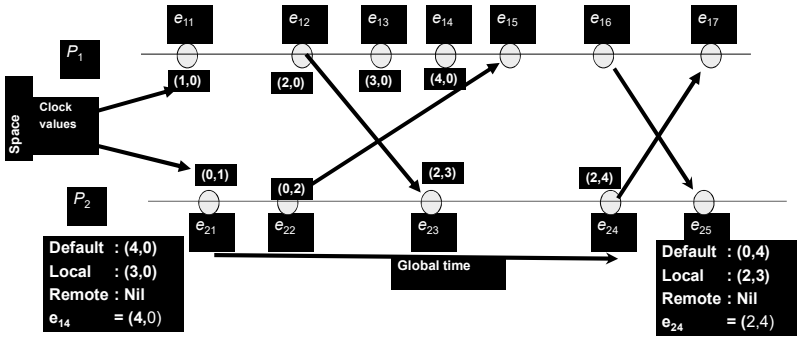
Vector Logical Clock Example



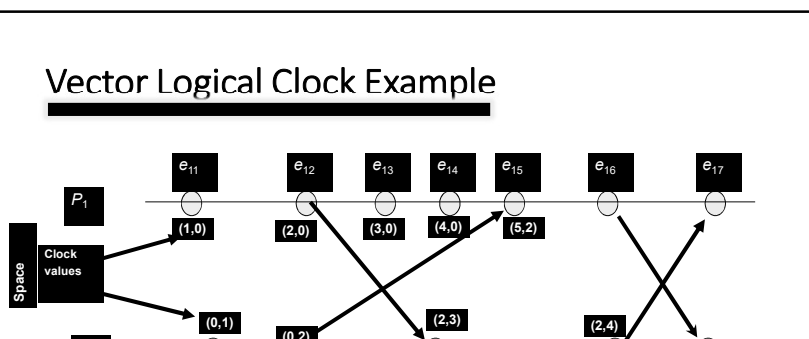
Vector Logical Clock Example



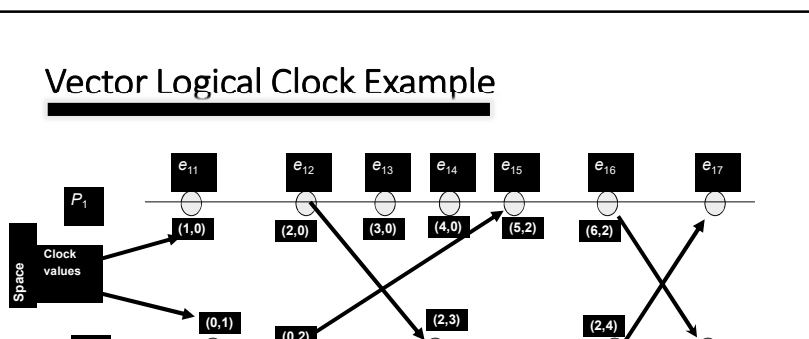
Vector Logical Clock Example



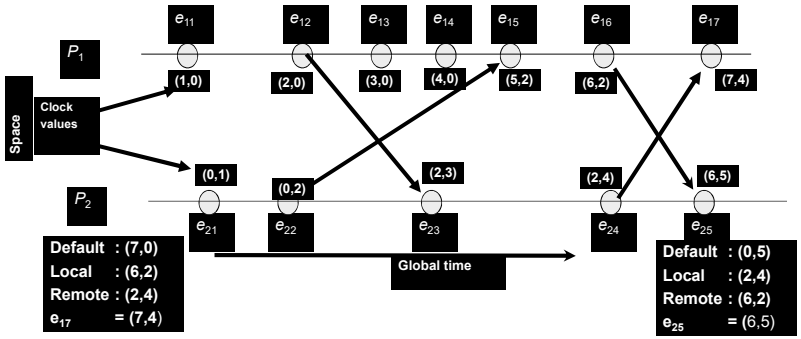
Vector Logical Clock Example



Vector Logical Clock Example

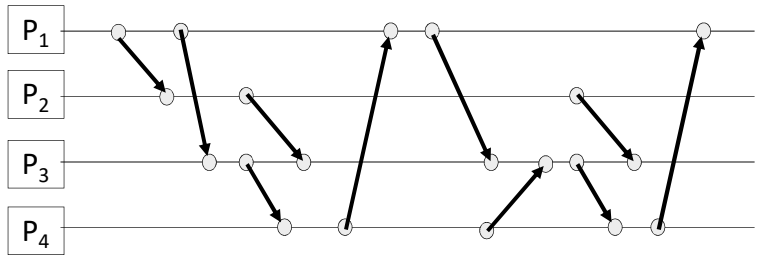


Vector Logical Clock Example



Other Examples

- ....



Query Regarding Lecture

Email: mnaseem105@gmail.com

Thanks

Email: mnaseem105@gmail.com